

EX.NO:1a

ANALYSIS OF FUNDAMENTAL NETWORK DIAGNOSTIC COMMANDS

DATE:

Aim: To study the different networking commands

- tcpdump
- netstat
- ifconfig
- nslookup and
- Traceroute

Description**1.tcpdump**

- tcpdump is a most powerful and widely used command-line packets sniffer or package analyzer tool which is used to capture or filter TCP/IP packets that received or transferred over a network on a specific interface.
- It is available under most of the Linux/Unix based operating systems.
- tcpdump also gives us a option to save captured packets in a file for future analysis.
- It saves the file in a pcapformat, that can be viewed by tcpdump command or a open source GUI based tool called Wireshark (Network Protocol Analyzier) that reads tcpdump pcap format files.

Prerequisite to capture packet on windows:

- **Windump** – Its Windows version of tcpdump
- **WinCap** – it allows applications to capture and transmit network packets.
- **Wireshark** – to open the pcap file need Wireshark.

Steps

1. *Open the terminal*
2. *Install the tcpdump (if not installed):*
sudo apt update
sudo apt install tcpdump
3. *List available network interfaces:*

ip link show

4. Capture packets on a specific interface

sudo tcpdump -i<ethernet-name>

5. Save captured packets to a file:

sudo tcpdump -i<ethernet-name>-w capture.pcap

6. Read a saved captured

file: **sudo tcpdump -r**

capture.pcap

Output:

- Packet details such as source IP, destination IP, protocol, and flags are displayed.
- Capture.pcap file is generated

```

t1ab2@t1ab2-HP-280-G4-MT-Business-PC:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether c4:65:16:1f:e9:0e brd ff:ff:ff:ff:ff:ff
t1ab2@t1ab2-HP-280-G4-MT-Business-PC:~$ sudo tcpdump -i enp3s0
tcpdump: verbose output suppressed, use -v|-vv for full protocol decode
listening on enp3s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
11:43:25.752295 STP 802.1w, Rapid STP, Flags [Learn, Forward, Agreement], bridge-id 8000.e4:a4:1c:58:9c:49.8004, length 36
11:43:25.961239 IP t1ab2-HP-280-G4-MT-Business-PC.39076 > b.resolvers.level3.net.domai: 63215 [Iau] A? connectivity-check.ubuntu.com. (58)
11:43:25.995717 IP b.resolvers.level3.net.domai > t1ab2-HP-280-G4-MT-Business-PC.39076: 63215 11/0/1 A 185.125.190.98, A 91.189.91.48, A 91.189.91.49, A 185.125.190.96, A 185.125.190.48, A 185.125.190.1
7, A 91.189.91.96, A 185.125.190.97 (234)
11:43:25.997532 IP t1ab2-HP-280-G4-MT-Business-PC.41004 > is-content-cache-2.ps5.canonical.com.http: Flags [S], seq 2176790740, win 64240, options [mss 1460,sackOK,TS val 2206507392,ecn 0,nop,wscale 7], length 0
11:43:25.999689 IP t1ab2-HP-280-G4-MT-Business-PC.51629 > b.resolvers.level3.net.domai: 31106* [Iau] PTR? 53.73.15.10.in-addr.arpa. (53)
11:43:26.004002 IP b.resolvers.level3.net.domai > t1ab2-HP-280-G4-MT-Business-PC.51629: 31106 NXDomain* 0/1/0 (112)
11:43:26.034336 IP t1ab2-HP-280-G4-MT-Business-PC.51629 > b.resolvers.level3.net.domai: 31106* PTR? 53.73.15.10.in-addr.arpa. (42)
11:43:26.068592 IP b.resolvers.level3.net.domai > t1ab2-HP-280-G4-MT-Business-PC.51629: 31106 NXDomain* 0/1/0 (101)
11:43:26.102165 IP t1ab2-HP-280-G4-MT-Business-PC.35284 > b.resolvers.level3.net.domai: 24809* [Iau] PTR? 18.190.125.185.in-addr.arpa. (50)
11:43:26.136380 IP b.resolvers.level3.net.domai > t1ab2-HP-280-G4-MT-Business-PC.35284: 24809 1/0/1 PTR is-content-cache-2.ps5.canonical.com. (106)
11:43:26.153750 IP is-content-cache-2.ps5.canonical.com.http > t1ab2-HP-280-G4-MT-Business-PC.41004: Flags [S], seq 2885642898, ack 2176790741, win 65535, options [mss 1460,sackOK,TS val 692522481,ecn 2206507392,nop,wscale 14], length 0
11:43:26.153812 IP t1ab2-HP-280-G4-MT-Business-PC.41004 > is-content-cache-2.ps5.canonical.com.http: Flags [S], ack 1, win 502, options [nop,nop,TS val 2206507548,ecn 692522481], length 0
11:43:26.154002 IP t1ab2-HP-280-G4-MT-Business-PC.41004 > is-content-cache-2.ps5.canonical.com.http: Flags [P], seq 1:80, ack 1, win 502, options [nop,nop,TS val 2206507548,ecn 692522481], length 87: HTTP: GET / HTTP/1.1
11:43:26.154314 IP is-content-cache-2.ps5.canonical.com.http > t1ab2-HP-280-G4-MT-Business-PC.41004: Flags [R], ack 88, win 3, length 0
^C
^C packets dropped by kernel
t1ab2@t1ab2-HP-280-G4-MT-Business-PC:~$ sudo tcpdump -i enp3s0 -w capture.pcap
tcpdump: listening on enp3s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C112 packets captured
112 packets received by filter
0 packets dropped by kernel
t1ab2@t1ab2-HP-280-G4-MT-Business-PC:~$ ls
.  .a.out  CHAPI  datahandling.ipynb  Documents  file.txt  oslab2.txt  Pictures  sin.c  snap  unix.c  Untitled2.ipynb  Untitled.ipynb  WineQT.csv
11:43:26.068592 capture.pcap  client.java  desktop  Downloads  Music  oslab.txt  server.java  simul.c  unix2.c  Untitled1.ipynb  Untitled3.ipynb  Videos
t1ab2@t1ab2-HP-280-G4-MT-Business-PC:~$ sudo tcpdump -r capture.pcap
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
11:44:23.750883 STP 802.1w, Rapid STP, Flags [Learn, Forward, Agreement], bridge-id 8000.e4:a4:1c:58:9c:49.8004, length 36
11:44:23.920662 IP 10.15.73.181.35777 > 239.255.255.250.1900: UDP, length 173
11:44:24.260309 STP 802.1w, Rapid STP, Flags [Learn, Forward], bridge-id 8625.9c:66:97:62:a1:20.8014, length 42
11:44:24.288038 IP 10.15.73.179.36702 > 239.255.255.250.1900: UDP, length 172
11:44:24.921904 IP 10.15.73.181.35777 > 239.255.255.250.1900: UDP, length 173
11:44:25.290518 IP 10.15.73.179.36702 > 239.255.255.250.1900: UDP, length 172
11:44:25.740003 STP 802.1w, Rapid STP, Flags [Learn, Forward, Agreement], bridge-id 8000.e4:a4:1c:58:9c:49.8004, length 36

```

2. netstat

- The netstat command, meaning network statistics, is a Command Prompt command used to display very detailed information about how our computer is communicating with other computers or network devices.
- Specifically, the netstat command can show details about individual network connections, overall and protocol-specific networking statistics, and much more, all of which could help troubleshoot certain kinds of networking issues.
- Netstat provides statistics for the following:
 - **Proto** - The name of the protocol (TCP or UDP).

Local Address - The IP address of the local computer and the port number being used. The name of the local computer that corresponds to the IP address and the name of the port is shown unless the -n parameter is specified. If the port is not yet established, the port number is shown as an asterisk (*).

○ **Foreign Address** - The IP address and port number of the remote computer to which the socket is connected. The names that correspond to the IP address and the port are shown unless the -n parameter is specified. If the port is not yet established, the port number is shown as an asterisk (*).

- Indicates the state of a TCP connection. The possible states are as follows:

State	Description
CLOSED	Indicates that the server has received an ACK signal from the client and the connection is Closed
CLOSE_WAIT	Indicates that the server has received the first FIN signal from the client and the connection is in the process of being closed
ESTABLISHED	Indicates that the server received the SYN signal from the client and the session is Established
FIN_WAIT_1	Indicates that the connection is still active but not currently being used
FIN_WAIT_2	Indicates that the client just received acknowledgment of the first FIN signal from the Server
LAST_ACK	Indicates that the server is in the process of sending its own FIN signal
LISTENING	Indicates that the server is ready to accept a connection
SYN_RECEIVED	Indicates that the server just received a SYN signal from the client
SYN_SEND	Indicates that this particular connection is open and active

Syntax: netstat [-a] [-e] [-n] [-o] [-p Protocol] [-r] [-s] [Interval]

EXAMPLES:

Used without parameters	Displays the IP address, subnet mask, and default gateway for all adapters.
-a	Displays all active TCP connections and the TCP and UDP ports on which the computer is Listening
-e	Displays Ethernet statistics, such as the number of bytes and packets sent and received. This parameter can be combined with -s.
-n	Displays active TCP connections, however, addresses and port numbers are expressed numerically and no attempt is made to determine names.
-o	Displays active TCP connections and includes the process ID (PID) for each connection. You can find the application based on the PID on the Processes tab in Windows Task Manager. This parameter can be combined with -a, -n, and -p.
-p	Shows connections for the protocol specified by Protocol. In this case, the Protocol can be tcp, udp, tcpv6, or udpv6. If this parameter is used with -s to display statistics by protocol, Protocol can be tcp, udp, icmp, ip, tcpv6, udpv6, icmpv6, or ipv6.
-s	Displays statistics by protocol. By default, statistics are shown for the TCP, UDP, ICMP, and IP protocols. If the IPv6 protocol for Windows XP is installed, statistics are shown for the TCP over IPv6, UDP over IPv6, ICMPv6, and IPv6 protocols. The -p parameter can be used to specify a set of protocols.
-r	Displays the contents of the IP routing table. This is equivalent to the route print Command.
Interval	Redisplays the selected information every Interval seconds. Press CTRL+C to stop the redisplay. If this parameter is omitted, netstat prints the selected information only once.
/?	- Displays help at the command prompt.

PROCEDURE:*CCI408-Networking Laboratory**Department of CSE (Cyber Security)**2025-2026*

1. Install net-tools package:

`sudo apt install net-tools`

2. Display all active connections:

`netstat -a`

3. Display numerical addresses and ports:

`netstat -an`

4. Show listening ports:

`netstat -l`

5. Display routing table:

`netstat -r`

6. Display interface statistics:

`netstat -e`**Output:**

3. ipconfig

1. Displays all current TCP/IP network configuration values and refreshes Dynamic Host Configuration Protocol (DHCP) and Domain Name System (DNS) settings.
2. This command is most useful on computers that are configured to obtain an IP address automatically.
3. This enables users to determine which TCP/IP configuration values have been configured by DHCP, Automatic Private IP Addressing (APIPA), or an alternate configuration.
4. If the Adapter name contains any spaces, use quotation marks around the adapter name (that is, "AdapterName").
5. For adapter names, ipconfig supports the use of the asterisk (*) wildcard character to specify either adapters with names that begin with a specified string or adapters with names that contain a specified string.
6. For example, Local* matches all adapters that start with the string Local and *Con* matches all adapters that contain the string Con.

Syntax

```
ipconfig [/all] [/renew [Adapter]] [/release [Adapter]] [/flushdns] [/displaydns] [/registerdns] [/shwclassid  
Adapter] [/setclassid Adapter [ClassID]]
```

PROCEDURE:

1. Install net-tools

```
sudo apt install net-tools
```

2. Display all network interfaces:

```
ifconfig
```

3. Display a specific interface:

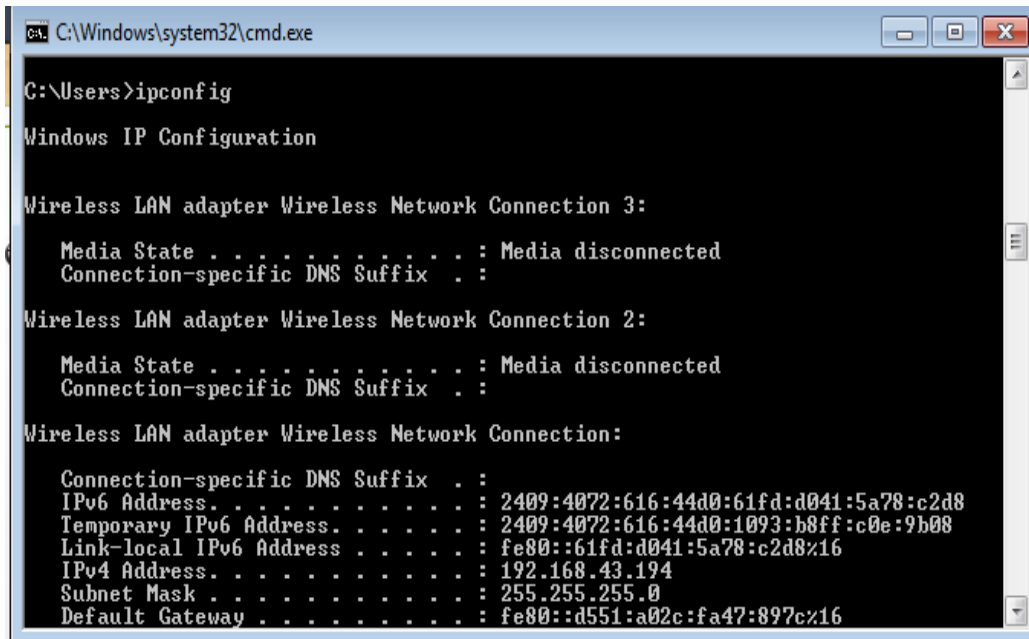
```
ifconfig<ethernet-name>
```

NOTE: ip addr show is the modern replacement but ifconfig is used for lab purposes

Parameters

Used without parameters	displays the IP address, subnet mask, and default gateway for all adapters.
/all	Displays the full TCP/IP configuration for all adapters.
/renew [Adapter]	Renews DHCP configuration for all adapters (if an adapter is not specified) or included.
/release [Adapter]	Sends a DHCPRELEASE message to DHCP server to release the current DHCP configuration and discard the IP address configuration for either all adapters (if an adapter is not specified) or for a specific adapter if the adapter Parameter is included.
/flushdns	Flushes and resets the contents of the DNS client resolver cache.
/displaydns	Displays the contents of the DNS client resolver cache, which includes both entries preloaded from the local Hosts file and any recently obtained resource records for name queries resolved by the computer.
/registerdns	Initiates manual dynamic registration for the DNS names and IP addresses that are configured at a computer.
/showclassid	Adapter Displays the DHCP class ID for a specified adapter. To see the DHCP class ID for all adapters, use the asterisk (*) wildcard character in place of Adapter. This parameter is available only on computers with adapters that are configured to obtain an IP address automatically.
/setclassid	Adapter [ClassID] Configures the DHCP class ID for a specified adapter. To set the DHCP class ID for all adapters, use the asterisk (*) wildcard character in place of Adapter.

OUTPUT



```
C:\Windows\system32\cmd.exe

C:\Users>ipconfig

Windows IP Configuration

Wireless LAN adapter Wireless Network Connection 3:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wireless Network Connection 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wireless Network Connection:

    Connection-specific DNS Suffix  . :
    IPv6 Address. . . . . : 2409:4072:616:44d0:61fd:d041:5a78:c2d8
    Temporary IPv6 Address. . . . . : 2409:4072:616:44d0:1093:b8ff:c0e:9b08
    Link-local IPv6 Address . . . . . : fe80::61fd:d041:5a78:c2d8%16
    IPv4 Address. . . . . : 192.168.43.194
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::d551:a02c:fa47:897c%16
```

ipconfig - To display the basic TCP/IP configuration for all adapters

ipconfig /all - To display the full TCP/IP configuration for all adapters

ipconfig /renew "Local Area - To renew a DHCP-assigned IP address configuration for only the
Connection" Local Area Connection adapter

ipconfig /flushdns - To flush the DNS resolver cache when troubleshooting DNS name resolution problems

ipconfig /showclassid Local - To display the DHCP class ID for all adapters with names that start with
Local

ipconfig /setclassid "Local Area - To set the DHCP class ID for the Local Area Connection adapter to
Connection" TEST TEST

4. nslookup

- nslookup is the name of a program that lets an Internet server administrator or any computer user enter a host name (for example, "whatis.com") and find out the corresponding IP address.
- It will also do reverse name lookup and find the host name for an IP address you specify.

Description	Command
Finding The IP Address of an Host	nslookup www.google.com
Reverse Lookup IP address to domain name	nslookup IP address nslookup 82.165.119.51
Find Mail Servers for a Domain	slookup -querytype=mx domain name nslookup -querytype=mx www.google.com

PROCEDURE (nslookup):

1. Find IP addresses of a domain:
nslookup www.google.com
2. Reverse DNS lookup:
nslookup 8.8.8.8
3. Find mail servers for a domain
nslookup -query=mx google.com

OUTPUT:

```
itlab2@itlab2-HP-280-G4-MT-Business-PC:~$ nslookup www.google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.154.119
Name:   www.google.com
Address: 142.251.152.119
Name:   www.google.com
Address: 142.251.157.119
Name:   www.google.com
Address: 142.251.151.119
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 142.251.153.119
Name:   www.google.com
Address: 2404:6800:4009:803::2004

itlab2@itlab2-HP-280-G4-MT-Business-PC:~$ nslookup 8.8.8.8
8.8.8.8.in-addr.arpa  name = dns.google.

Authoritative answers can be found from:

itlab2@itlab2-HP-280-G4-MT-Business-PC:~$ nslookup -query=mx google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
google.com    mail exchanger = 10 smtp.google.com.

Authoritative answers can be found from:
```

5. TRACEROUTE:

Traceroute is a network diagnostic tool that displays the route taken by packets across a network and measures any transit delays.

Syntax: tracert domain name
Example: tracert google.com
 tracert 64.13.192.208

PROCEDURE:

1. Install traceroute
sudo apt install traceroute
2. Trace route to a domain:
traceroute google.com
3. Trace route to an IP address:
traceroute 8.8.8.8

OUTPUT:

```
itlab2@itlab2-HP-280-G4-MT-Business-PC:~$ traceroute google.com
traceroute to google.com (142.251.43.14), 30 hops max, 60 byte packets
 1 _gateway (10.15.73.1) 0.321 ms 0.240 ms 0.244 ms
 2 192.168.1.2 (192.168.1.2) 0.187 ms 0.172 ms 0.155 ms
 3 static-37.132.195.14-tataidc.co.in (14.195.132.37) 2.489 ms 2.919 ms 2.557 ms
 4 10.117.225.13 (10.117.225.13) 5.449 ms 4.315 ms 4.194 ms
 5 142.250.171.162 (142.250.171.162) 2.318 ms 2.460 ms 2.477 ms
 6 * * *
 7 108.170.231.128 (108.170.231.128) 2.014 ms 142.250.224.6 (142.250.224.6) 2.394 ms 142.251.55.236 (142.251.55.236) 2.927 ms
 8 142.250.62.66 (142.250.62.66) 3.015 ms 142.251.230.70 (142.251.230.70) 7.236 ms 142.250.208.230 (142.250.208.230) 18.296 ms
 9 216.239.49.131 (216.239.49.131) 19.959 ms 142.251.252.95 (142.251.252.95) 23.033 ms 142.251.241.173 (142.251.241.173) 26.487 ms
10 192.178.254.228 (192.178.254.228) 23.388 ms 192.178.254.238 (192.178.254.238) 25.916 ms 192.178.254.240 (192.178.254.240) 25.751 ms
11 142.250.226.135 (142.250.226.135) 23.607 ms 192.178.111.61 (192.178.111.61) 26.376 ms 142.250.213.101 (142.250.213.101) 23.835 ms
12 142.251.77.101 (142.251.77.101) 23.835 ms 142.251.77.99 (142.251.77.99) 21.401 ms 142.251.77.101 (142.251.77.101) 20.800 ms
13 tsa03s08-in-f14.1e100.net (142.251.43.14) 22.498 ms 19.730 ms 19.229 ms
itlab2@itlab2-HP-280-G4-MT-Business-PC:~$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1 _gateway (10.15.73.1) 0.525 ms 0.448 ms 0.384 ms
 2 192.168.1.2 (192.168.1.2) 0.146 ms 0.167 ms 0.149 ms
 3 static-37.132.195.14-tataidc.co.in (14.195.132.37) 2.333 ms 2.379 ms 2.706 ms
 4 10.117.225.13 (10.117.225.13) 3.171 ms 3.047 ms 3.122 ms
 5 142.250.171.162 (142.250.171.162) 2.308 ms 2.641 ms 2.289 ms
 6 * * *
 7 dns.google (8.8.8.8) 2.581 ms 3.115 ms 3.306 ms
```

Result

Thus, a study on different networking commands were made successfully

EX.NO:1b & 1c ROUTE PATH ANALYSIS USING TRACEROUTE UTILITY

DATE:

AIM:

To capture and analyze **Ping** and **Traceroute** PDUs using **Wireshark** in Ubuntu Linux.

PROCEDURE**Step 1: Start Wireshark Capture**

1. Open **Terminal**.
2. Start Wireshark using: `sudo wireshark`
3. Select the active network interface (e.g., `eth0` or `wlan0`).
4. Click **Start Capture**.

Step 2: Generate Traceroute Traffic

1. Open a new **Terminal** window.
2. Execute the traceroute command: `traceroute -n 8.8.8.8`
(`-n` disables DNS resolution to reduce traffic)
3. After completion, close the terminal.
4. Stop the capture in Wireshark.

Step 3: Analyze ICMP Packets

1. In Wireshark, apply the filter: `icmp`
2. Select an **ICMP Echo Request** packet.
3. Expand:
 - **Internet Protocol Version 4**
 - Observe **Time To Live (TTL)** value.
 - **Internet Control Message Protocol**
 - **Type = 8 (Echo Request)**
4. Select the corresponding **ICMP Time Exceeded** packet.
5. Observe:
 - **Type = 11 (Time Exceeded)**
 - **Code = 0 (TTL exceeded in transit)**

- Source IP (router where TTL expired)
6. Repeat observation for subsequent ICMP packets

OUTPUT

The screenshot shows a Wireshark capture of ICMP packets. The packet list pane displays several packets, with packet 16 selected. The packet details pane shows the structure of the selected packet, which is an ICMP Time Exceeded message. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
14	60.26	192.168.1.10	192.168.1.1	ICMP	84	Echo (ping) request id=99811, seq=1, ttl=1
15	60.87	192.168.1.10	203.50.113.1	ICMP	84	Time Exceeded (Time to Live exceeded)
16	18.83	192.168.1.10	203.0.111.1	ICMP	84	Echo Exceeded (Time to Live exceeded)
29	13.28	192.168.1.10	298.51.100.2	ICMP	84	Echo (ping) request id=99811, seq=1, ttl=2
22	13.29	192.168.1.10	203.0.113.2	ICMP	84	Time Exceeded (Time to Live exceeded)
23	13.24	192.168.1.10	103.0.1.131	ICMP	94	Echo (ping) request id=99811, seq=2, ttl=2
26	12.25	192.168.1.10	112.0.2.45	ICMP	84	Time Exceeded (Time to Live exceeded)
27	13.26	192.168.1.10	8.8.8.8	ICMP	84	Echo (ping) request id=29811, seq=4, ttl=3
28	13.27	192.168.1.10	198.51.100.2	ICMP	84	Time Exceeded (Time to Live exceeded)
29	13.28	192.168.1.10	812.0.4.5	ICMP	84	Echo (ping) request id=29811, seq=4, ttl=4
20	23.26	192.168.1.10	8.8.8.8	ICMP	84	Echo Exceeded (Time to Live exceeded)

Internet Protocol Version 4, Src: 198.51.100.2, Dst: 192.168.1.10

- Time to Live: 1
- Internet Control Message Protocol
 - Type: 11 (Time Exceeded)
 - Code: 0 (Time to Live exceeded in transit)
 - Checksum: 0xd9ce [correct]
 - Sender IP address: 198.51.100.2
 - Original IP datagram

Packet 16 details (hex):

```

352 65 05 00 00 50 61 70 00 01 01 05 00 00 05 66 66 08 00 E..T. VS/.,...
352 63 03 01 51 56 58 87 70 07 00 56 01 01 10 12 c3 01 01 V1.11 .,5...
362 109 06 08 00 50 a8 a1 a8 01 0a ..... ~..
  
```

ICMP Time: (icmp)

Packet: 29 Packets: 82 Displayed: 18 (21.95%) Profile: Default 0-Execns Received 27 of 82 (33.8%)

- ICMP Echo Request packets captured with varying TTL values
- ICMP Time Exceeded replies from intermediate routers
- Destination reached when TTL is sufficient

RESULT

Ping and Traceroute PDUs were successfully captured and analyzed using Wireshark on Ubuntu Linux. ICMP Echo Request and Time Exceeded

EX.NO:2 DEVELOPMENT OF HTTP SOCKET FOR WEB PAGETRANSFER(UPLOAD/DOWNLOAD)**DATE:****AIM:**

To write and execute an HTTP web client program using TCP sockets in Python on **Ubuntu Linux** to download a web page from a web server.

ALGORITHM

1. Open the terminal in Ubuntu Linux.
2. Create a TCP socket using the `socket()` function.
3. Obtain the server IP address using the domain name.
4. Establish a connection to the web server on port 80.
5. Send an HTTP GET request to the server.
6. Receive the response (HTML content) from the server.
7. Display the received web page content.
8. Close the socket connection.

PROGRAM

```
import socket

# Create a TCP socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to the web server
client_socket.connect(("www.aurcc.ac.in", 80))

# HTTP GET request
request = "GET / HTTP/1.1\r\nHost: www.aurcc.ac.in\r\n\r\n"

client_socket.sendall(request.encode())

# Receive response from server response =

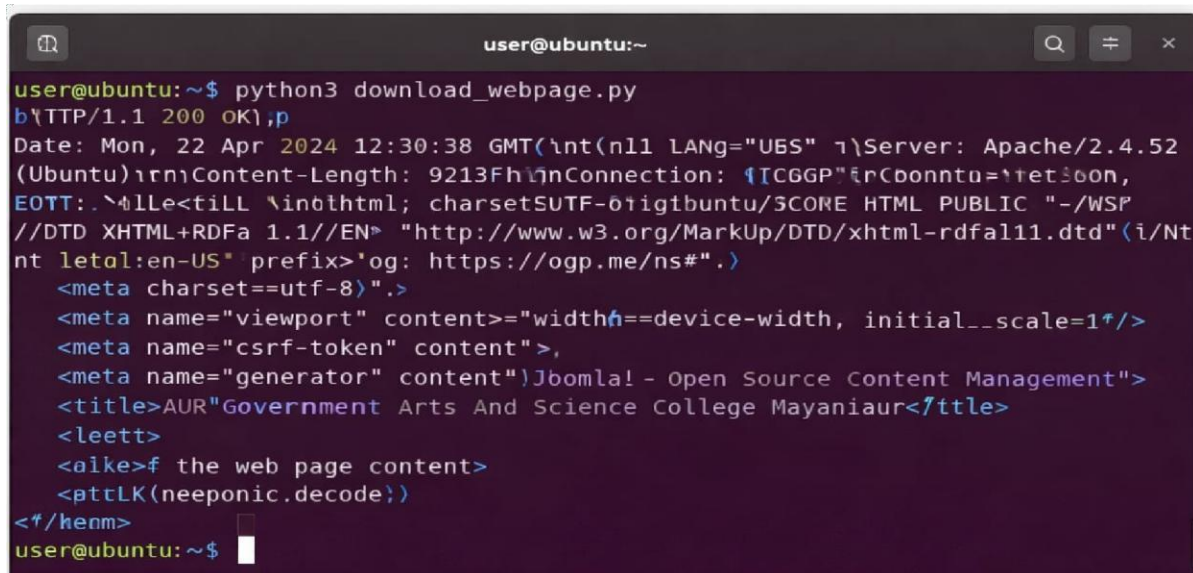
client_socket.recv(4096)

# Display the web page content print(response.decode())

# Close the socket client_socket.close()
```

OUTPUT

The HTML source code of the requested web page (`www.aurcc.ac.in`) is displayed on the terminal.



```
user@ubuntu:~$ python3 download_webpage.py
b'\HTTP/1.1 200 OK';p
Date: Mon, 22 Apr 2024 12:30:38 GMT(\nt(nl1 LANG="UBS" \Server: Apache/2.4.52
(Ubuntu)\rnContent-Length: 9213Fh\InConnection: \ICGGP"i\rCbonntu=\tet:6on,
E0TT:\`4ll<fiLL \inbhtml; charsetSUTF-6iigibuntu/3CORE HTML PUBLIC "-//WSP
//DTD XHTML+RDFa 1.1//EN> "http://www.w3.org/MarkUp/DTD/xhtml-rdfa11.dtd"<i/Nt
nt letal:en-US" prefix>'og: https://ogp.me/ns#".>
<meta charset==utf-8>".>
<meta name="viewport" content>="widthf==device-width, initial__scale=1f/>
<meta name="csrf-token" content">,
<meta name="generator" content")Jbomla! - Open Source Content Management">
<title>AUR"Government Arts And Science College Mayaniaur</title>
<leett>
<alke>f the web page content>
<pttLK(neeponic.decode;)
</kenm>
user@ubuntu:~$
```

RESULT

Thus, the HTTP web client program to download a web page using TCP sockets was successfully executed on **Ubuntu Linux**, and the web page content was received and displayed correctly.

EX.NO: 2b**TCP SOCKET APPLICATION -ECHO CLIENT AND ECHO SERVER****DATE:****AIM**

To write a Java program for ECHO using TCP Sockets

ALGORITHM**Client-Side Programming**

- **Establish a Socket Connection**

1. To connect to other machine, we need a socket connection.
2. A socket connection means the two machines have information about each other's network location (IP Address) and TCP port.
3. The java.net.Socket class represents a Socket.
4. To open a socket: **Socket socket = new Socket ("127.0.0.1", 5000)**
5. First argument – IP address of Server. (127.0.0.1 is the IP address of localhost, where code will run on single stand-alone machine).
6. Second argument – TCP Port. (Just a number representing which application to run on a server. for example, HTTP runs on port 80. Port number can be from 0 to 65535)

- **Communication**

1. To communicate over a socket connection, streams are used to both input and output the data.

- **Closing the connection**

1. The socket connection is closed explicitly once the message to server is sent.
2. In the program, Client keeps reading input from user and sends to the server until "Over" is typed.

Client Program:

```
import java.net.*;
import java.io.*;
public class Client
{
    // initialize socket and input output streams
    private Socket socket = null;
    private DataInputStream input = null;
    private DataOutputStream out = null;
    public Client(String address, int port)
    {
        // establish a connection

        try
        {
            socket = new Socket(address, port);
            System.out.println("Connected");
            // takes input from terminal
            input = new DataInputStream(System.in);
            // sends output to the socket

            out = new DataOutputStream(socket.getOutputStream());
```

```
}
catch(UnknownHostException u)
{
System.out.println(u);
}
catch(IOException i)
{
System.out.println(i);
}
// string to read message from input String
line = "";
while (!line.equals("Over"))
{
try
{
line = input.readLine();
out.writeUTF(line);
}
catch(IOException i)
{
System.out.println(i);
}
}
try
{
input.close();
out.close();
socket.close();
}
catch(IOException i)
{
System.out.println(i);
}
}
public static void main(String args[])
{
Client client = new Client("127.0.0.1", 5000);
} }
```

Server Programming

• Establish a Socket Connection

- To write a server application two sockets are needed.
- A ServerSocket which waits for the client requests (when a client makes a new Socket())
- A plain old Socket socket to use for communication with the client.

• Communication

- getOutputStream() method is used to send the output through the socket.

• Close the Connection

- After finishing, it is important to close the connection by closing the socket as well as input/output streams.

Server Program:

```
import java.net.*;
import java.io.*;
public class Server
{
//initialize socket and input stream
private Socket      socket = null;
private ServerSocket server = null;
private DataInputStream in      = null;
// constructor with port public
Server(int port)
{
// starts server and waits for a connection try
{
server = new ServerSocket(port);
System.out.println("Server started");

System.out.println("Waiting for a client ...");
socket = server.accept();
System.out.println("Client accepted");
//      takes input from the client socket
in = new DataInputStream(new BufferedInputStream(socket.getInputStream())); String
line = "";
//      reads message from client until "Over" is sent
while (!line.equals("Over"))
{

try
{
line = in.readUTF(); System.out.println(line);
}
catch(IOException i)
{
System.out.println(i);
}
}
System.out.println("Closing connection");

//      close connection
socket.close(); in.close();
}
catch(IOException i)
{
}
```

```

System.out.println(i);
}
}
public static void main(String args[])
{
Server server = new Server(5000);
}}

```

OUTPUT

Open two windows one for Server and another for Client

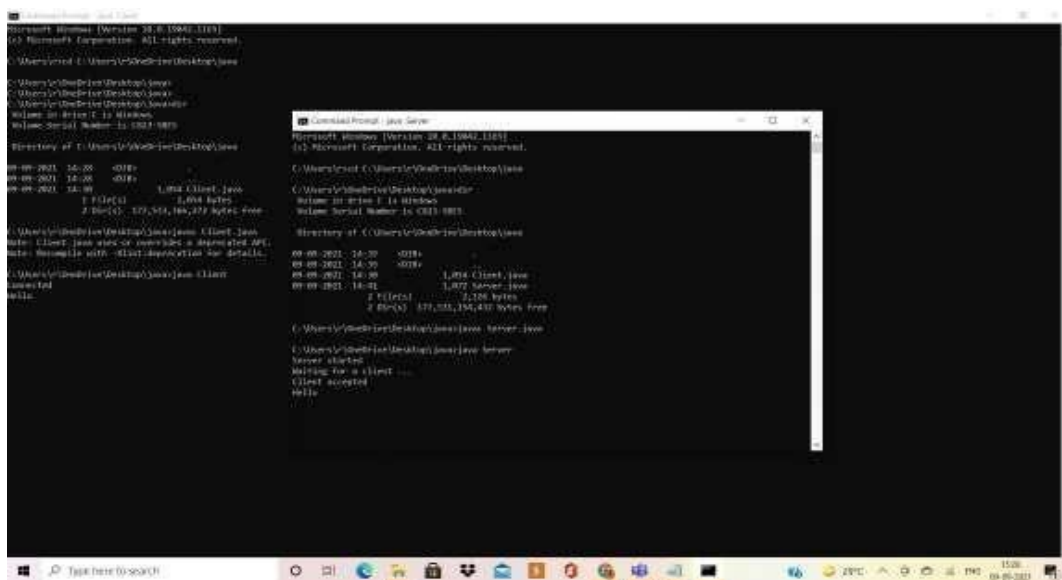
1. First run the Server application as , \$ java Server Server started Waiting for a client ...
2. Then run the Client application on another terminal as, \$ java Client It will show – Connected and the server accepts the client and shows, Client accepted
3. Then start typing messages in the Client window. Here is a sample input to the Client Hello

I made my first socket connection Over

Which the Server simultaneously receives and shows, Hello I made my first socket connection Over

Closing connection

Sending "Over" closes the connection between the Client and the Server just like said before.



RESULT

Thus the data from client to server is echoed using TCP connection

EX NO : 2c**CHAT APPLICATION USING TCP SOCKETS****DATE :****AIM**

To write a java program to initiate a chat between client and server.

ALGORITHM**CLIENT**

1. Start the program
2. Include necessary package in java
3. Create a socket in client to server.
4. The client establishes a connection to the server.
5. The client accepts the connection and to send the data from client to server.
6. The client communicates the server to send the end of the message
7. Stop the program.

SERVER

1. Start the program
2. Include necessary package in java
3. Create a socket in server to client
4. The server establishes a connection to the client.
5. The server accepts the connection and to send the data from server to client and vice versa
6. The server communicates the client to send the end of the message.
7. Stop the program.

PROGRAM**TCPserver1.java**

```
import java.net.*;
import java.io.*;
public class tcpserver
{
    public static void main(String arg[])
    {
        ServerSocket s=null;
        String line;
        DataInputStream is=null,is1=null;
        PrintStream os=null;
        Socket c=null;
        try
        {
            s= new ServerSocket(9999);
        }
        catch(IOException e)
        {
            System.out.println(e);
        }
        try
        {
            c=s.accept();
```

```
is=new DataInputStream(c.getInputStream());
is1=new DataInputStream(System.in);
os=new PrintStream(c.getOutputStream());
do
{
line=is.readLine();
System.out.println("Client:"+line);
System.out.println("Server:");
line=is1.readLine();
os.println(line);
}
while(line.equalsIgnoreCase("quit")==false);
is.close();
os.close();
}
catch(IOException e)
{
System.out.println(e);
}
}
}
}
TCPClient1.java
import java.net.*;
import java.io.*;
public class tcpclient
{
public static void main(String arg[])
{
Socket c=null;
String line;
DataInputStream is,is1;
PrintStream os;
try
{
c=new Socket( "127.0.0.1",9999);
}
catch(IOException e)
{
System.out.println(e);
}
try
{
os=new PrintStream(c.getOutputStream());
is=new DataInputStream(System.in);
is1=new DataInputStream(c.getInputStream());
do
{
System.out.println("Client:");
line=is.readLine();
os.println(line);
System.out.println("Server:" + is1.readLine());
}
while(line.equalsIgnoreCase("quit")==false);
is1.close();
os.close();
}
```

```
catch(IOException e)
```

```
{  
System.out.println("Socket Closed!Message Passing is over");  
}  
}  
}
```

OUTPUT:

Server Console

```
$ javac tcpchatserver.java
```

```
$ java tcpchatserver
```

Server started

Client connected

Server <<< hi

Message to Client : hello

Server <<< how r u?

Message to Client : fine

Server <<< me too

Message to Client : bye

Client Disconnected

Client Console

```
$ javac tcpchatclient.java
```

```
$ java tcpchatclient
```

Type "end" to Quit

Message to Server : hi

Client <<< hello

Message to Server : how r u?

Client <<< fine

Message to Server : me too

Client <<< bye

Message to Server : end

RESULT

Thus, the CHAT program was successfully executed.

EX.NO: 2d**FILE TRANSFER USING TCP SOCKETS****DATE:****AIM**

To create a socket (TCP) between two computers and enable file transfer between them.

ALGORITHM**Server**

1. Start the program.
2. A file named **Received** which is automatically created used to store the file contents.
3. Create a socket with address family AF_INET, type SOCK_STREAM and default protocol.
4. Initialize the socket and set its attributes assign any port number as desired.
5. Bind the server to the socket using bind() function.
6. Establish the listen queue using the listen function.
7. Wait for client program on request, establish a connection using accept function.
8. Repeat steps 8-10 until the server sends bye.
9. Read the message from the client using read() function. display the message.
10. If the client message is bye go to step 11.
11. Accept the server message and write it to client using write() function.
12. Close the connection.
13. Stop the program.

Client Program:

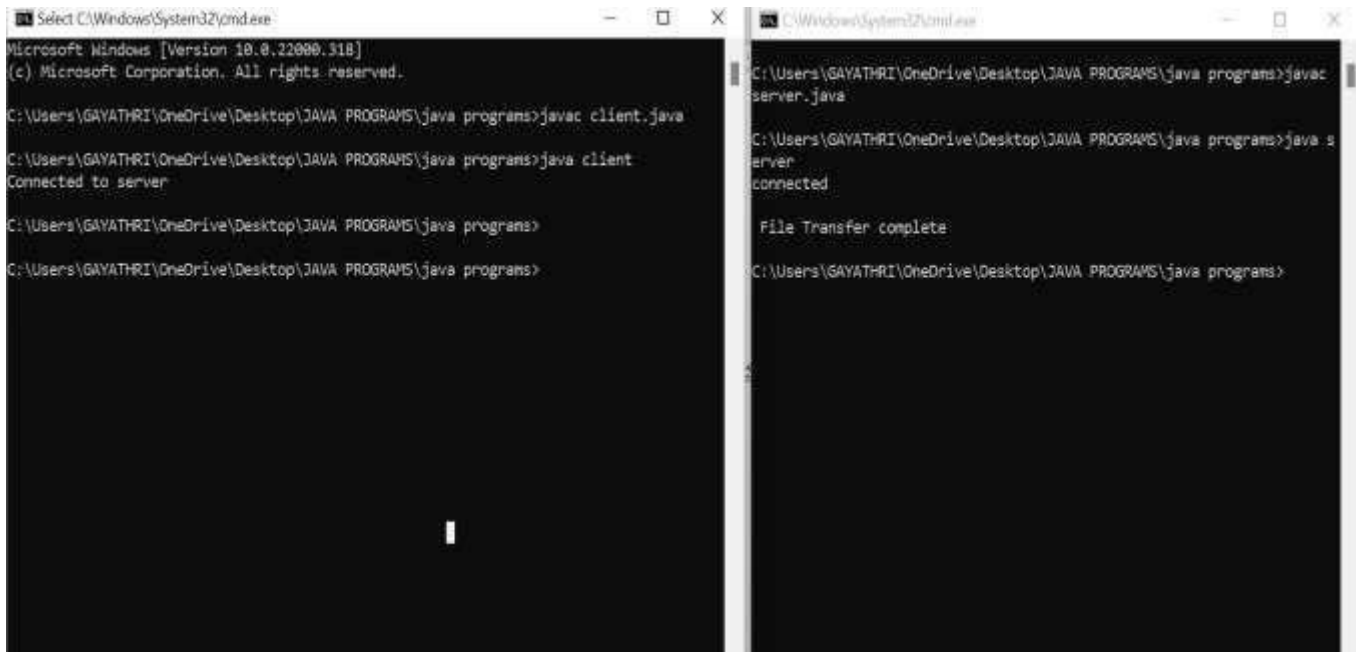
1. Start the program.
 2. Create a text file named **Send** and type some contents for which we want to transfer.
 3. Create a socket with address family AF_INET type SOCK_STREAM and default protocol.
 4. Initialize the socket and set its attributes set the required port number.
 5. Connect to the server using connect () function to initialize the request.
 6. Repeat steps 6-8 until server sends bye
 7. Accept the client message and write it to the server using write function.
 8. If the client message is bye go to step 9.
 9. Read the message from the server using read() function.
 10. Display the message.
- Stop the program.

Client Program

```
import java.io.*;
import java.net.*;
class client
{
    public static void main(String args[])throws Exception
    {
        Socket s=new Socket("127.0.0.1",9999);
        if(s.isConnected())
        {
            System.out.println("Connected to server");
            FileOutputStream fout = new FileOutputStream("received.txt");
            DataInputStream din = new DataInputStream(s.getInputStream());
            int r;
            while((r=din.read())!=-1)
            {
                fout.write((char)r);
            }
            s.close();
        }
    }
}
```

Server Program:

```
import java.io.*;
import java.net.*;
class server
{
    public static void main(String args[])throws Exception
    {
        ServerSocket ss=new ServerSocket(9999);
        Socket s=ss.accept();
        System.out.println("connected");
        FileInputStream fin = new FileInputStream("send.txt");
        DataOutputStream dout = new DataOutputStream(s.getOutputStream());
        int r;
        while((r=fin.read())!=-1)
        {
            dout.write(r);
        }
        System.out.println("\n File Transfer complete");
        s.close();
        ss.close();
    }
}
```

OUTPUT:

```
Microsoft Windows [Version 10.0.22000.318]
(c) Microsoft Corporation. All rights reserved.

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>javac client.java

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>java client
Connected to server

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>

C:\Windows\System32\cmd.exe

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>javac
server.java

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>java s
erver
connected

File Transfer complete

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS\java programs>
```

RESULT:

Thus the File Transfer Protocol using TCP was successfully executed

Exp. No: 2e**SIMULATION OF DNS USING UDP SOCKETS****Date:****AIM**

To create a client server program to Domain Name System using the UDP protocol client server.

ALGORITHM**Server**

Step1: Start the program.

Step2: Create the socket for the server. Step3:

Bind the socket to the port.

Step4: Listen for the incoming client connection.

Step5: Receive the IP address from the client to be resolved. Step6:

Get the domain name for the client.

Step7: Check the existence of the domain in the server.

Step8: If domain matches then send the corresponding address to the client. Step9:

Stop the program execution

Client

Step1: Start the Program.

Step2: Create the socket for the client. Step3:

Connect the socket to the Server.

Step4: Send the host name to the server to be resolved.

Step5: If the server corresponds then print the address and terminate the process

PROGRAM**UDPclient** import

java .io.*; import

java.net.*;

classUDPclient

{

public static DatagramSocket ds;

public static intclientport=789,serverport=790;

public static void main(String args[])throws Exception

{

byte buffer[]=new byte[1024]; ds=new

DatagramSocket(serverport);

BufferedReader dis=new BufferedReader(new InputStreamReader(System.in));

System.out.println("server waiting");

InetAddressia=InetAddress.getLocalHost();

while(true)

{

System.out.println("Client:");

String str=dis.readLine(); if(str.equals("end"))

buffer=str.getBytes();

ds.send(new DatagramPacket(buffer,str.length(),ia,clientport));

DatagramPacket p=new DatagramPacket(buffer,buffer.length);

ds.receive(p);

```
String psx=new String(p.getData(),0,p.getLength());
System.out.println("Server:" + psx);
}
}
}
```

UDP server

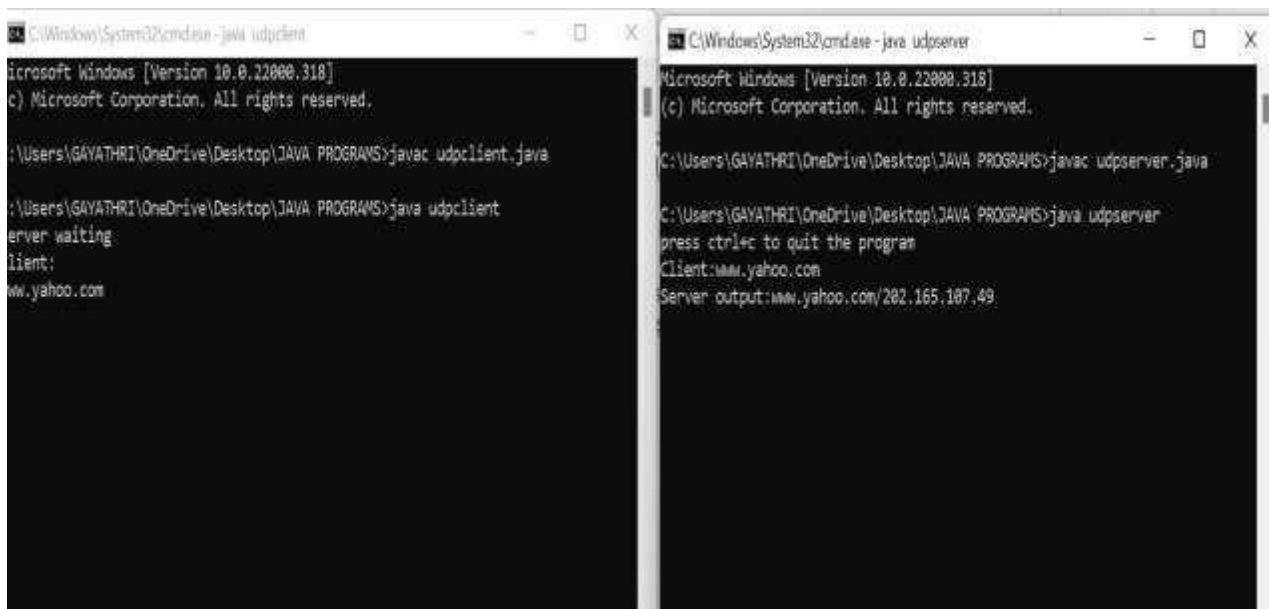
```
import java.io.*;
import java.net.*;
classUDPserver
{
    public static DatagramSocket ds;
    public static byte buffer[]=new byte[1024]; public
    static intclientport=789,serverport=790;
    public static void main(String args[])throws Exception
    {
        ds=new DatagramSocket(clientport);
        System.out.println("press ctrl+c to quit the program");
        BufferedReader dis=new BufferedReader(new InputStreamReader(System.in)); InetAddressia=InetAddress.getLocalHost();
        while(true)
        {
            DatagramPacket p=new DatagramPacket(buffer,buffer.length);
            ds.receive(p);
            String psx=new String(p.getData(),0,p.getLength());
            System.out.println("Client:" + psx);
            InetAddressib=InetAddress.getByName(psx);
            System.out.println("Server output:"+ib);
            String str=dis.readLine();
            if(str.equals("end")) break;
            buffer=str.getBytes();
            ds.send(new DatagramPacket(buffer,str.length(),ia,serverport));
        }
    }
}
```

OUTPUT**UDPclient**

```
D:\Program Files\Java\jdk1.6.0\bin>javac UDPclient.java
D:\Program Files\Java\jdk1.6.0\bin>java UDPclient Server
waiting
Client:www.yahoo.com
```

UDPserver

```
D:\Program Files\Java\jdk1.6.0\bin>javac UDPserver.java
D:\Program Files\Java\jdk1.6.0\bin>java UDPserver
Press ctrl+c to quit the program
Client:www.yahoo.com
Server output:www.yahoo.com/106.10.170.115
```

Output:

The image shows two side-by-side screenshots of Windows command prompts. The left window, titled 'C:\Windows\System32\cmd.exe - java udpclient', shows the execution of 'javac udpclient.java' and 'java udpclient Server'. The output shows 'waiting' and 'Client:www.yahoo.com'. The right window, titled 'C:\Windows\System32\cmd.exe - java udpserver', shows the execution of 'javac udpserver.java' and 'java udpserver'. The output shows 'press ctrl+c to quit the program', 'Client:www.yahoo.com', and 'Server output:www.yahoo.com/202.165.107.49'.

```
C:\Windows\System32\cmd.exe - java udpclient
Microsoft Windows [Version 10.0.22000.318]
(c) Microsoft Corporation. All rights reserved.

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS>javac udpclient.java

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS>java udpclient
Server waiting
Client:
www.yahoo.com

C:\Windows\System32\cmd.exe - java udpserver
Microsoft Windows [Version 10.0.22000.318]
(c) Microsoft Corporation. All rights reserved.

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS>javac udpserver.java

C:\Users\GAYATHRI\OneDrive\Desktop\JAVA PROGRAMS>java udpserver
press ctrl+c to quit the program
Client:www.yahoo.com
Server output:www.yahoo.com/202.165.107.49
```

RESULT

Thus client server program to Domain Name System using the UDP protocol client server has been executed and verified successfully

Exp. No: 3a**Date:****SIMULATING ARP PROTOCOL USING JAVA****AIM**

To simulate ARP protocol using JAVA language.

ALGORITHM

Step 1: Import the net, input output, language, utility package s

Step 2: Use the method exec with the command “arp -a “ as parameter.

Step 3: Store the result of the ping command in input stream. Step 4:

Process the input stream and display the result.

Step 5: End the process

PROGRAM

```
import java.net.*;
import java.io.*;
import java.lang.Object; // O Should be a capital import
import java.util.*;
class arp
{
    public static void main(String args[])
    {
        try
        {
            Process p=Runtime.getRuntime().exec("arp -a");
            Buffered Reader br=new BufferedReader(new InputStreamReader(p.getInputStream())); String
            str,str1="",st1,st2;
            while((str=br.readLine())!=null)
            {
                str1+=str+"\n";
                StringTokenizer st=new StringTokenizer(str1,"\n");
                BufferedReader br1=new BufferedReader(new InputStreamReader(System.in)); System.out.println("Enter
                the IP ADDRESS");
                st2=br1.readLine();
                while(st.hasMoreTokens())
                {
                    st1=st.nextToken();
                    if(st1.indexOf(st2)!=-1)
                    System.out.println(st1);
                }
            }
        }
        catch(Exception E)
        {
            E.printStackTrace();
        }
    }
}
```

OUTPUT

Enter the IP ADDRESS: 172.15.150.110 00-
01-0C-33-1C-5D

RESULT

Thus the ARP protocol is successfully completed

Exp. No: 3a	SIMULATING RARP PROTOCOL USING JAVA
Date:	

AIM

To stimulate RARP protocol using JAVA language.

ALGORITHM

Step 1: Import the net, input output, language, utility packages.

Step 2: Use the method exec with the command “arp -a “as parameter.

Step 3: Store the result of the ping command in input stream. Step 4:

Process the input stream and display the result.

Step 5: End the process.

PROGRAM

```
import java.net.*;
import java.io.*;
import java.lang.Object; // O Should be a capital import
java.util.*;
class rarp
{
public static void main(String args[])
{
try
{
Process p=Runtime.getRuntime().exec("arp -a");
BufferedReader br=new BufferedReader(new InputStreamReader(p.getInputStream())); String
str,str1="",st1,st2;
while((str=br.readLine())!=null)
str1+=str+"\n";
StringTokenizer st=new StringTokenizer(str1,"\n");
BufferedReader br1=new BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter the physical 48-bit ADDR ENTER THE PHYSICAL 48BIT ADDRESS");
st2=br1.readLine();
while(st.hasMoreTokens())
{
st1=st.nextToken();
if(st1.indexOf(st2)!=-1)
System.out.println(st1);
}
}
catch(Exception E)
{
E.printStackTrace();
}
}
```

OUTPUT

ENTER THE PHYSICAL 48BIT ADDRESS: 00-01-0C-33-1C-5D
172.15.150.110

RESULT

Thus the RARP protocols are successfully completed.

Exp. No: 3b	STUDY OF NETWORK SIMULATOR (NS)
Date:	

AIM

To study about NS2 simulator in detail.

THEORY

Network Simulator (Version 2), widely known as NS2, is simply an event driven simulation tool that has proved useful in studying the dynamic nature of communication networks. Simulation of wired as well as wireless network functions and protocols (e.g., routing algorithms, TCP, UDP) can be done using NS2. In general, NS2 provides users with a way of specifying such network protocols and simulating their corresponding behaviors. Due to its flexibility and modular nature, NS2 has gained constant popularity in the networking research community since its birth in 1989. Ever since, several revolutions and revisions have marked the growing maturity of the tool, thanks to substantial contributions from the players in the field. Among these are the University of California and Cornell University who developed the REAL network simulator,¹ the foundation which NS is based on. Since 1995 the Defense Advanced Research Projects Agency (DARPA) supported development of NS through the Virtual Inter Network Testbed (VINT) project. Currently the National Science Foundation (NSF) has joined the ride in development. Last but not the least, the group of Researchers and developers in the community are constantly working to keep NS2 strong and versatile.

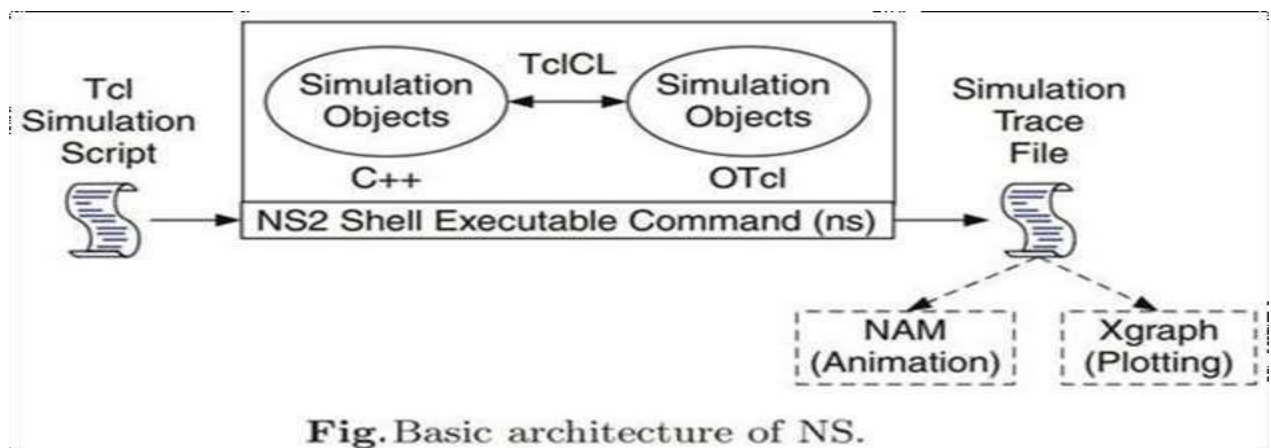
BASIC ARCHITECTURE

Figure shows the basic architecture of NS2. NS2 provides users with an executable command ns which takes on input argument, the name of a Tcl simulation scripting file. Users are feeding the name of a Tcl simulation script (which sets up a simulation) as an input argument of an NS2 executable command ns.

In most cases, a simulation trace file is created, and is used to plot graph and/or to create animation. NS2 consists of two key languages: C++ and Object-oriented Tool Command Language (OTcl). While the C++ defines the internal mechanism (i.e., a backend) of the simulation objects, the OTcl sets up simulation by assembling and configuring the objects as well as scheduling discrete events (i.e., a frontend).

The C++ and the OTcl are linked together using TclCL. Mapped to a C++ object, variables in the OTcl domains are sometimes referred to as handles. Conceptually, a handle (e.g., n as a Node handle) is just a string (e.g., _o10) in the OTcl domain, and does not contain any functionality. Instead, the functionality (e.g., receiving a packet) is defined in the mapped C++ object (e.g., of class Connector). In the OTcl domain, a handle acts as a frontend which interacts with users and other OTcl objects. It may define its own procedures and variables to facilitate the interaction. Note that the member procedures and variables in the OTcl domain are called instance procedures (instprocs) and instance variables (instvars), respectively. Before proceeding further, the readers are encouraged to learn C++ and OTcl languages. We refer the readers to [14] for the detail of C++, while a brief tutorial of Tcl and OTcl tutorial are given in Appendices A.1 and A.2 respectively.

NS2 provides a large number of built-in C++ objects. It is advisable to use these C++ objects to set up a simulation using a Tcl simulation script. However, advance users may find these objects insufficient. They need to develop their own C++ objects, and use a OTcl configuration interface to put together these objects. After simulation, NS2 outputs either text-based or animation-based simulation results. To interpret these results graphically and interactively, tools such as NAM (Network AniMator) and XGraph are used. To analyze a particular behavior of the network, users can extract a relevant subset of text-based data and transform it to a more conceivable presentation.

CONCEPT OVERVIEW

NS uses two languages because simulator has two different kinds of things it needs to do. On one hand, detailed simulations of protocols require a systems programming language which can efficiently manipulate bytes, packet headers, and implement algorithms that run over large data sets. For these tasks run-time speed is important and turn-around time (run simulation, find bug, fix bug, recompile, re-run) is less important. On the other hand, a large part of network research involves slightly varying parameters or configurations, or quickly exploring a number of scenarios.

In these cases, iteration time (change the model and re-run) is more important. Since configuration runs once (at the beginning of the simulation), run-time of this part of the task is less important. ns meets both of these needs with two languages, C++ and OTcl.

Tcl scripting

Tcl is a general purpose scripting language. [Interpreter]

- Tcl runs on most of the platforms such as Unix, Windows, and Mac.
- The strength of Tcl is its simplicity.
- It is not necessary to declare a data type for variable prior to the usage.

Basics of TCL

Syntax: command arg1 arg2 arg3

Hello World!

```
puts stdout {Hello, World!} Hello, World!
```

Variables Command Substitution

```
set a 5 set len [string length foobar]
set b $a set len [expr [string length foobar] +
```

Wired TCL Script Components

Create the event scheduler
 Open new files & turn on the tracing
 Create the nodes
 Setup the links
 Configure the traffic type (e.g., TCP, UDP, etc) Set the
 time of traffic generation (e.g., CBR, FTP) Terminate
 the simulation

NS Simulator Preliminaries.

1. Initialization and termination aspects of the ns simulator.
2. Definition of network nodes, links, queues and topology.
3. Definition of agents and of applications.
4. The nam visualization tool.
5. Tracing and random variables.

Initialization and Termination of TCL Script in NS-2

An ns simulation starts with the command

set ns [new Simulator]

Which is thus the first line in the tcl script. This line declares a new variable as using the set command, you can call this variable as you wish, In general people declares it as ns because it is an instance of the Simulator class,so an object the code[new Simulator] is indeed the installation of the class Simulator using the reserved word new.In order to have output files with data on the simulation (trace files) or files used for visualization (nam files), we need to create the files using —open command:

#Open the Trace file

set tracefile1 [open out.tr w]

\$ns trace-all

\$tracefile1 #Open the NAM

trace file

set namfile [open out.nam w]

\$ns namtrace-all \$namfile

The above creates a dta trace file called out.tr and a nam visualization trace file called out.nam. Within the tcl script, these files are not called explicitly by their names, but instead by pointers that are declared above and called —tracefile1 and —namfile respectively. Remark that they begins with a # symbol. The second line open the file —out.tr to be used for writing, declared with the letter —w. The third line uses a simulator method called trace-all that have as parameter the name of the file where the traces will go.

Define a “finish” procedure

Proc finish { } {

global ns tracefile1 namfile

\$ns flush-trace

Close

\$tracefile1

Close \$namfile

Exec nam out.nam &

Exit 0

}

Definition of a network of links and nodes

The way to define a node is

set n0 [\$ns node]

Once we define several nodes, we can define the links that connect them. An example of a definition of a link is:

\$ns duplex-link \$n0 \$n2 10Mb 10ms DropTail

Which means that \$n0 and \$n2 are connected using a bi-directional link that has 10ms of propagation delay and

a capacity of 10Mb per sec for each direction.

To define a directional link instead of a bi-directional one, we should replace —duplex-link by —simplex-link. In ns, an output queue of a node is implemented as a part of each link whose input is that node. We should also define the buffer capacity of the queue related to each link. An example would be:

```
#set Queue Size of link (n0-n2) to 20
$ns queue-limit $n0 $n2 20
```

FTP over TCP

TCP is a dynamic reliable congestion control protocol. It uses Acknowledgements created by the destination to know whether packets are well received.

There are number variants of the TCP protocol, such as Tahoe, Reno, NewReno, Vegas. The type of agent appears in the first line:

```
set tcp [new Agent/TCP]
```

The command **\$ns attach-agent \$n0 \$tcp** defines the source node of the tcp connection.

The command **set sink [new Agent /TCPSink]** Defines the behavior of the destination node of TCP and assigns to it a pointer called sink.

#Setup a UDP

```
connection set udp
```

```
[new Agent/UDP]
```

```
$ns attach-agent $n1
```

```
$udp set null [new
```

```
Agent/Null]
```

```
$ns attach-agent $n5 $null
```

```
$ns connect $udp $null
```

```
$udp set fid_2
```

#setup a CBR over UDP connection

The below shows the definition of a CBR application using a UDP agent The command **\$ns attach-agent \$n4 \$sink** defines the destination node.

The command **\$ns connect \$tcp \$sink**

finally makes the TCP connection between the source and destination nodes.

```
set cbr [new Application/Traffic/CBR]
```

```
$cbr attach-agent $udp
```

```
$cbr set packetSize_ 100
```

```
$cbr set rate_ 0.01Mb
```

```
$cbr set random_ false
```

TCP has many parameters with initial fixed defaults values that can be changed if mentioned explicitly. For example, the default TCP packet size has a size of 1000bytes. This can be changed to another value, say 552bytes, using the command **\$tcp set packetSize_ 552**.

When we have several flows, we may wish to distinguish them so that we can identify them with different colors in the visualization part. This is done by the command **\$tcp set fid_ 1** that assigns to the TCP connection a flow identification of —1. We shall later give the flow identification of —2 to the UDP connection.

RESULT

Thus the Network Simulator 2 is studied in detail.

Exp. No: 3c	STUDY OF UDP PERFORMANCE USING SIMULATION TOOL
Date:	

Aim

To study the performance of UDP by simulating a simple network

Algorithm

1. Create a simulator object
2. Define different color for data flows
3. Trace all events in a nam file.
4. Create four nodes $n0$, $n1$, $n2$ and $n3$
5. Describe their layout topology
6. Specify the link capacity between nodes
7. Monitor queue on the link $n2$ to $n3$ vertically 90°
8. Create a UDP agents $udp0$, $udp1$ and attach it to nodes $n0$ and $n1$ respectively
9. Create a CBR traffic $cbr0$, $cbr1$ and attach it to $udp0$ and $udp1$ respectively
10. Create a traffic sink and attach it to node $n3$
11. Connect sources to the sink
12. Label the nodes
13. Schedule $cbr0$ to start at 0.5 and stop at 4.5 seconds
14. Schedule $cbr1$ to start at 1.0 and stop at 4.0 seconds
15. Call finish procedure at 5.0 seconds
16. Run the simulation
17. Execute NAM on the trace file
18. Observe simulated events on the NAM and packet flow on link $n2$ to $n3$
19. Stop

Program

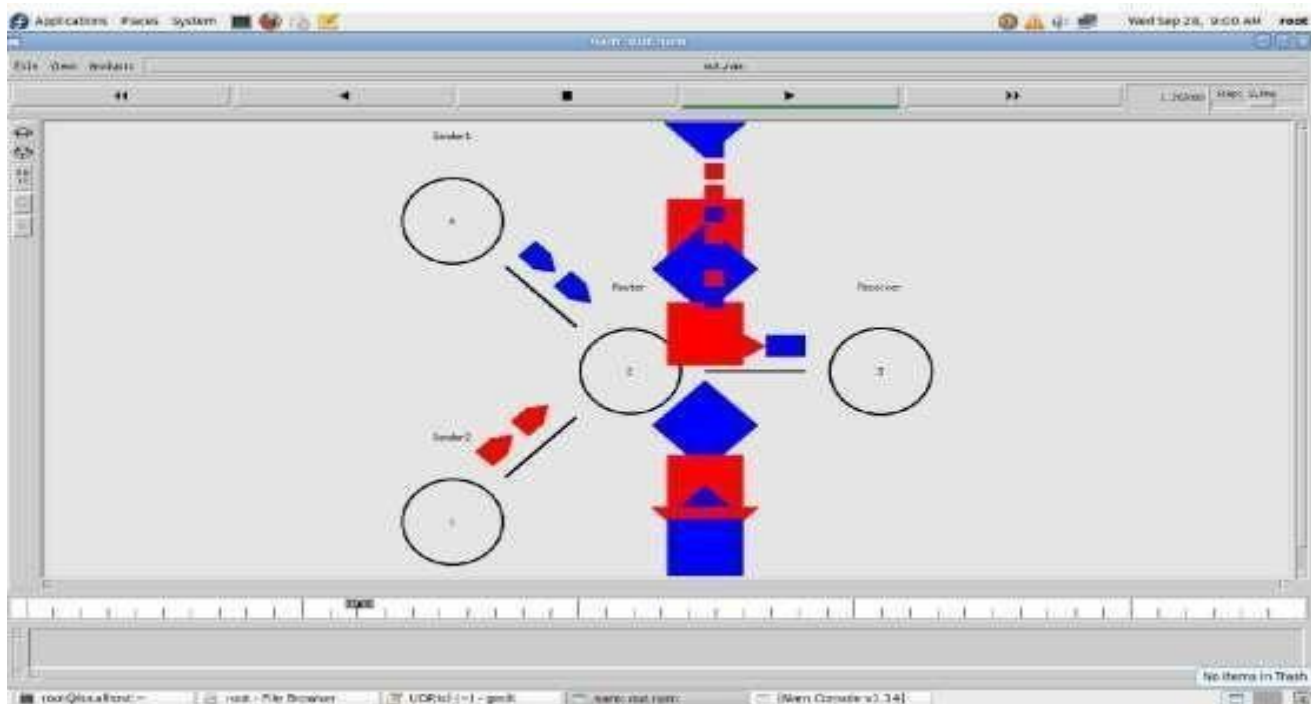
```
#Study of UDP performance -
UDP.tcl #Create a simulator object
set ns [new Simulator]
#Define different colors for data flows
$ns color 1 Blue
$ns color 2 Red
#Open the nam trace
file set nf [open
out.nam w]
$ns namtrace-all
$nf #Create four
nodes set n0 [$ns
node]
set n1 [$ns
node] set n2
[$ns node]
set n3 [$ns
node]
#Create links between the nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n3 $n2 1Mb 10ms
SFQ #Specify layout of nodes
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right
#Monitor the queue for the link 23
vertically
$ns duplex-link-op $n2 $n3 queuePos 0.5
#Create a UDP agent and attach it to node
n0 set udp0 [new Agent/UDP]
$udp0 set class_ 1
$ns attach-agent $n0 $udp0
# Create a CBR traffic source and attach it to
udp0 set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
#Create a UDP agent and attach it to node
n1 set udp1 [new Agent/UDP]
$udp1 set class_ 2
$ns attach-agent $n1 $udp1
# Create a CBR traffic source and attach it to
udp1 set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
#Create a Null agent (a traffic sink) and attach it to node
n3 set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
#Connect traffic sources with the traffic sink
```

```

$ns connect $udp0 $null0
$ns connect $udp1
$null0 #Define finish
procedure proc finish
{} {
global ns nf
#Close the trace file close $nf
#Execute nam on the trace
file exec nam -a out.nam
&
exit 0
}
#Define label for nodes
$ns at 0.0 "$n0 label Sender1"
$ns at 0.0 "$n1 label Sender2"
$ns at 0.0 "$n2 label Router"
$ns at 0.0 "$n3 label Receiver"
#Schedule events for the CBR agents
$ns at 0.5 "$cbr0 start"
$ns at 1.0 "$cbr1 start"
$ns at 4.0 "$cbr1 stop"
$ns at 4.5 "$cbr0 stop"
#Call finish procedure after 5 seconds of simulation time
$ns at 5.0
"finish" #Run
the simulation

```

Output



Result

Thus the performance of UDP and basic network terminologies were studied using NS2.

STUDY OF TCP PERFORMANCE USING SIMULATION TOOL**Exp. No: 3c****Date:**

Date:

Aim

To study the performance of a TCP network with droptail queue mechanism on the gateway

Algorithm

1. Create a simulator object
2. Define different flows for data flows
3. Trace all events in a nam file and text file
4. Create source nodes ($s1, s2, s3$), gateway (G) and receiver (r)
5. Describe their layout topology
6. Specify the link between nodes
7. Define the queue size between nodes G and r as 5
8. Monitor queue on all links vertically 90°
9. Create TCP agents $tcp1, tcp2, tcp3$ and attach it to nodes $s1, s2$ and $s3$ respectively
10. Create three TCP sinks and attach it to node r
11. Connect traffic sources to the sink
12. Create FTP agents $ftp1, ftp2, ftp3$ and attach it to $tcp1, tcp2$ and $tcp3$ respectively
13. Label the nodes at start time
14. Schedule $ftp1, ftp2, ftp3$ to start at 0.1 and stop at 5.0 seconds
15. Call *finish* procedure at 5.25 seconds
16. Run the simulation
17. Execute NAM on the trace file
18. Observe the simulated events on the NAM editor and packet flow on link G to r
19. View the trace file and analyse the events
20. Stop

Program

```
#Study of TCP performance -
TCP.tcl #Create a simulator object
set ns [new
Simulator] #Open
trace files
set f [open droptail-queue-out.tr w]
$ns trace-all $f
#Open the nam trace file
set nf [open droptail-queue-out.nam w]
$ns namtrace-all $nf
#s1, s2 and s3 act as
sources. set s1 [$ns node]
set s2 [$ns
node] set s3
[$ns node]
#G acts as a
gateway set G [$ns
node]
#r acts as a
receiver set r
[$ns node]
#Define different colors for data flows
$ns color 1 red
$ns color 2 SeaGreen
$ns color 3 blue
#Create links between the nodes
$ns duplex-link $s1 $G 6Mb 10ms DropTail
$ns duplex-link $s2 $G 6Mb 10ms DropTail
$ns duplex-link $s3 $G 6Mb 10ms DropTail
$ns duplex-link $G $r 3Mb 10ms
DropTail #Define the layout of the nodes
$ns duplex-link-op $s1 $G orient right-up
$ns duplex-link-op $s2 $G orient right
$ns duplex-link-op $s3 $G orient right-down
$ns duplex-link-op $G $r orient right
#Define the queue size for the link between node G and r
$ns queue-limit $G $r 5
#Monitor the queues for links vertically
$ns duplex-link-op $s1 $G queuePos 0.5
$ns duplex-link-op $s2 $G queuePos 0.5
$ns duplex-link-op $s3 $G queuePos 0.5
$ns duplex-link-op $G $r queuePos 0.5

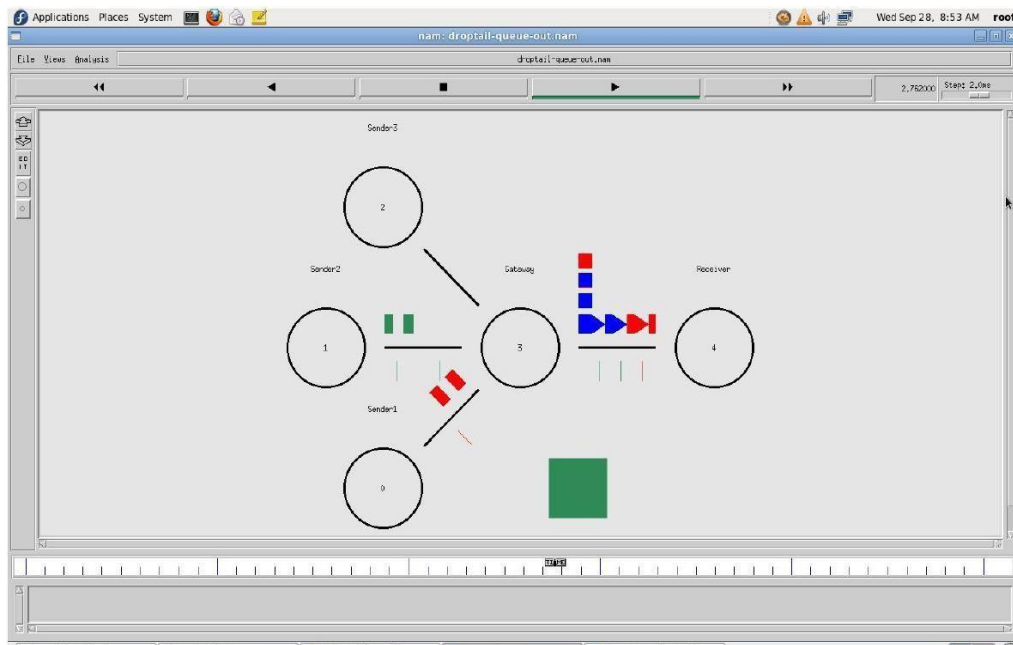
#Create a TCP agent and attach it to node
s1 set tcp1 [new Agent/TCP/Reno]
$ns attach-agent $s1 $tcp1
$tcp1 set window_ 8
$tcp1 set fid_ 1
#Create a TCP agent and attach it to node
s2 set tcp2 [new Agent/TCP/Reno]
```

```
$ns attach-agent $s2 $tcp2
$tcp2 set window_ 8
$tcp2 set fid_ 2

#Create a TCP agent and attach it to node
s3 set tcp3 [new Agent/TCP/Reno]
$ns attach-agent $s3 $tcp3
$tcp3 set window_ 4
$tcp3 set fid_ 3
#Create TCP sink agents and attach them to node
r set sink1 [new Agent/TCPSink]
set sink2 [new
Agent/TCPSink] set sink3
[new Agent/TCPSink]
$ns attach-agent $r $sink1
$ns attach-agent $r $sink2
$ns attach-agent $r $sink3
#Connect the traffic sources with the traffic sinks
$ns connect $tcp1 $sink1
$ns connect $tcp2 $sink2
$ns connect $tcp3 $sink3
#Create FTP applications and attach them to
agents set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1
set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2
set ftp3 [new Application/FTP]
$ftp3 attach-agent $tcp3
#Define a 'finish'
procedure proc finish {} {
global ns
$ns flush-trace
puts "running nam..."
exec nam -a droptail-queue-out.nam
& exit 0
}
#Define label for nodes
$ns at 0.0 "$s1 label Sender1"
$ns at 0.0 "$s2 label Sender2"
$ns at 0.0 "$s3 label Sender3"
$ns at 0.0 "$G label Gateway"
$ns at 0.0 "$r label Receiver"
#Schedule ftp events
$ns at 0.1 "$ftp1 start"
$ns at 0.1 "$ftp2 start"
$ns at 0.1 "$ftp3 start"
$ns at 5.0 "$ftp1 stop"
$ns at 5.0 "$ftp2 stop"
$ns at 5.0 "$ftp3 stop"
#Call finish procedure after 5 seconds of simulation time
$ns at 5.25
"finish" #Run
```

the simulation
\$ns run

Output



RESULT

Thus the performance of TCP and basic network terminologies were studied using NS2

Exp. No: 4a	SIMULATION OF DISTANCE VECTOR ROUTING ALGORITHM – RIP
Date:	

AIM

To simulate and study the Distance Vector Routing algorithm – RIP using NS-2 simulation.

THEORY

Distance Vector Routing is one of the routing algorithms in a Wide Area Network for computing shortest path between source and destination. The Router is one main device used in a wide area network. The main task of the router is Routing. It forms the routing table and delivers the packets depending upon the routes in the table- either directly or via an intermediate device.

Each router initially has information about its all neighbors. Then this information will be shared among nodes.

ALGORITHM

1. Create a simulator object
2. Define different colors for different data flows
3. Open a nam trace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose distance vector routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

PROGRAM

```
#Create a simulator
object set ns [new
Simulator]
#Use distance vector routing
$ns rtproto DV
#Open the nam trace
file set nf [open
out.nam w]
$ns namtrace-all
$nf # Open
tracefile
set nt [open trace.tr w]
$ns trace-all $nt
#Define 'finish'
procedure proc finish
{} {
global ns nf
$ns flush-trace
#Close the trace
file close $nf
```

```
#Execute nam on the trace
```

```
file exec nam -a out.nam
```

```
&
```

```
exit 0
```

```
}
```

```
# Create 8
```

```
nodes set n1
```

```
[$ns node]
```

```
set n2 [$ns
```

```
node] set n3
```

```
[$ns node]
```

```
set n4 [$ns
```

```
node] set n5
```

```
[$ns node]
```

```
set n6 [$ns
```

```
node] set n7
```

```
[$ns node]
```

```
set n8 [$ns
```

```
node]
```

```
# Specify link characteristics
```

```
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
```

```
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
```

```
$ns duplex-link $n5 $n6 1Mb 10ms DropTail
```

```
$ns duplex-link $n6 $n7 1Mb 10ms DropTail
```

```
$ns duplex-link $n7 $n8 1Mb 10ms DropTail
```

```
$ns duplex-link $n8 $n1 1Mb 10ms
```

```
DropTail # specify layout as a octagon
```

```
$ns duplex-link-op $n1 $n2 orient left-up
```

```
$ns duplex-link-op $n2 $n3 orient up
```

```
$ns duplex-link-op $n3 $n4 orient right-up
```

```
$ns duplex-link-op $n4 $n5 orient right
```

```
$ns duplex-link-op $n5 $n6 orient right-down
```

```
$ns duplex-link-op $n6 $n7 orient down
```

```
$ns duplex-link-op $n7 $n8 orient left-down
```

```
$ns duplex-link-op $n8 $n1 orient left
```

```
#Create a UDP agent and attach it to
```

```
node n1 set udp0 [new Agent/UDP]
```

```
$ns attach-agent $n1 $udp0
```

```
#Create a CBR traffic source and attach it to
```

```
udp0 set cbr0 [new Application/Traffic/CBR]
```

```
$cbr0 set packetSize_ 500
```

```
$cbr0 set interval_ 0.005
```

```
$cbr0 attach-agent $udp0
```

```
#Create a Null agent (a traffic sink) and attach it to node
```

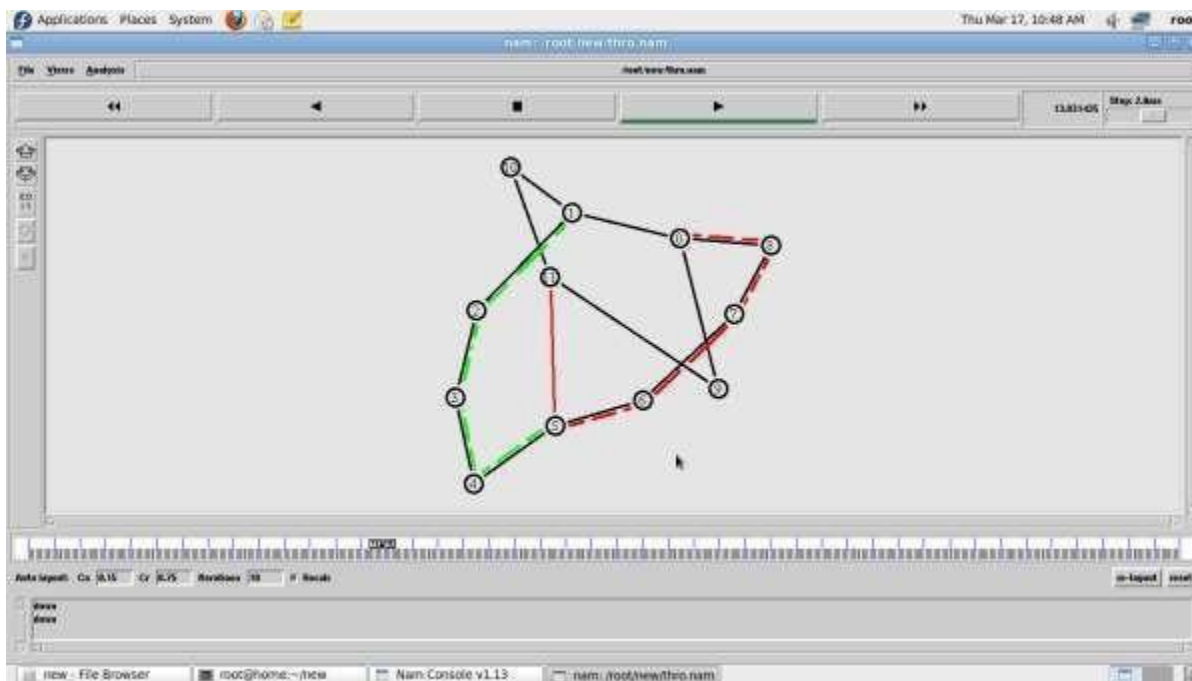
```
n4 set null0 [new Agent/Null]
```

```
$ns attach-agent $n4 $null0
```

```
#Connect the traffic source with the traffic sink
```

```
$ns connect $udp0 $null0
#Schedule events for the CBR agent and the network dynamics
$ns at 0.0 "$n1 label Source"
$ns at 0.0 "$n4 label Destination"
$ns at 0.5 "$cbr0 start"
$ns rtmodel-at 1.0 down $n3 $n4
$ns rtmodel-at 2.0 up $n3 $n4
$ns at 4.5 "$cbr0 stop"
#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0
"finish" #Run
the simulation
$ns run
```

OUTPUT



RESULT

Thus the Distance vector Routing Algorithm was Simulated and studied.

Exp. No: 4b	SIMULATION OF LINK STATE ROUTING ALGORITHM
Date:	

AIM

To simulate and study the link state routing algorithm using NS-2 simulation.

THEORY

In **link state routing**, each router shares its knowledge of its neighborhood with every other router in the internet work. (i) **Knowledge about Neighborhood:** Instead of sending its entire routing table a router sends info about its neighborhood only. (ii) **To all Routers:** each router sends this information to every other router on the internet work not just to its neighbor. It does so by a process called **flooding**. (iii) **Information sharing when there is a change:** Each router sends out information about the neighbors when there is change.

ALGORITHM

1. Create a simulator object
2. Define different colors for different data flows
3. Open a nam trace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose Link state routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

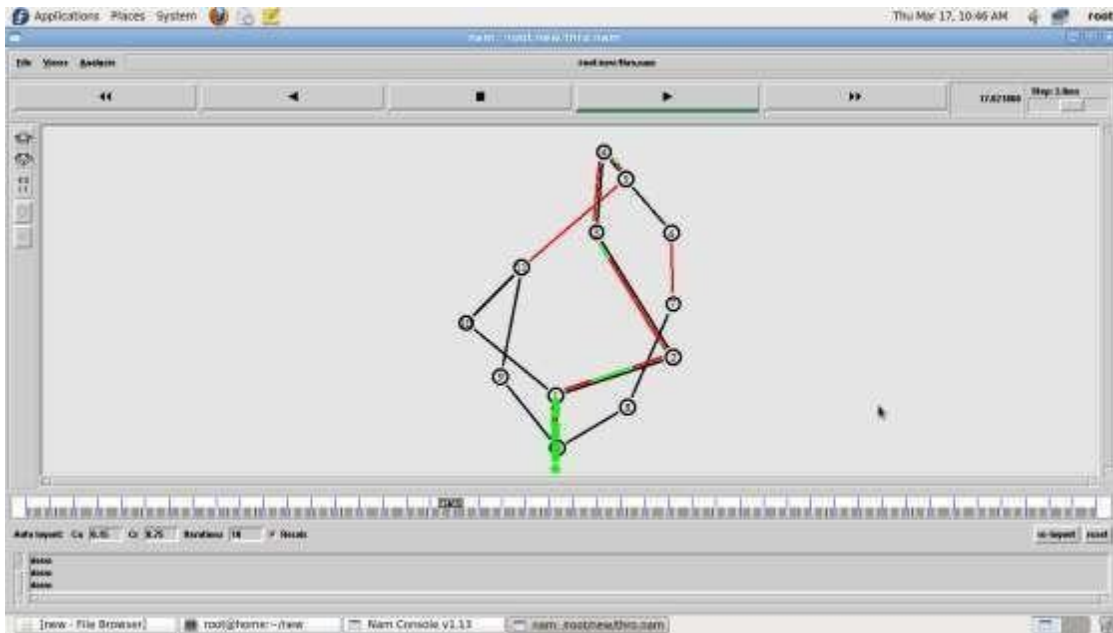
PROGRAM

```
set ns [new
Simulator] set nr
[open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all
$nf proc finish { }
{ global ns nr nf
$ns flush-
trace close
$nf
close $nr
exec nam thro.nam
& exit 0
}
for { set i 0 } { $i < 12 } { incr i 1
} { set n($i) [$ns node]}
for {set i 0 } { $i < 8 } {incr i} {
$ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms
DropTail set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$scr0 set packetSize_ 500
$scr0 set interval_ 0.005
$scr0 attach-agent
$udp0 set null0 [new
Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0
$null0 set udp1 [new
Agent/UDP]
$ns attach-agent $n(1) $udp1
set cbr1 [new Application/Traffic/CBR]
$scr1 set packetSize_ 500
$scr1 set interval_ 0.005
$scr1 attach-agent
$udp1 set null0 [new
Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0
$ns rtproto LS
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
```

```
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)
$udp0 set fid_1
$udp1 set fid_2
$ns color 1 Red
$ns color 2 Green
$ns at 1.0 "$cbr0 start"
$ns at 2.0 "$cbr1 start"
$ns at 45 "finish"
```

\$ns run

OUTPUT



RESULT

Thus, the Link State Routing Algorithm was Simulated and studied.

Exp. No: 4c	Performance Evaluation of Routing protocols using Simulation tool. UNICAST ROUTING PROTOCOL
Date:	

AIM:

To write a ns2 program for implementing unicast routing protocol.

ALGORITHM:

1. Start the program.
2. Declare the global variables ns for creating a new simulator.
3. Set the color for packets.
4. Open the network animator file in the name of file2 in the write mode.
5. Open the trace file in the name of file 1 in the write mode.
6. Set the unicast routing protocol to transfer the packets in network.
7. Create the required no of nodes.
8. Create the duplex-link between the nodes including the delay time,bandwidth and dropping queue mechanism.
9. Give the position for the links between the nodes.
10. Set a tcp reno connection for source node.
11. Set the destination node using tcp sink.
12. Setup a ftp connection over the tcp connection.
13. Down the connection between any nodes at a particular time.
14. Reconnect the downed connection at a particular time.
15. Define the finish procedure.
16. In the definition of the finish procedure declare the global variables ns, file1, and file2.
17. Close the trace file and name file and execute the network animation file.
18. At the particular time call the finish procedure.
19. Stop the program.

PROGRAM:

```
set ns [new Simulator]
#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
#Open the Trace file
set file1 [open out.tr w]
$ns trace-all $file1
#Open the NAM trace file
set file2 [open out.nam w]
$ns namtrace-all $file2
#Define a 'finish' procedure
proc finish { } {
    global ns file1 file2
    $ns flush-trace
    close $file1
    close $file2
    exec nam out.nam &
    exit 0
}
# Next line should be commented out to have the static routing
$ns rtproto DV
#Create six nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
#Create links between the nodes
$ns duplex-link $n0 $n1 0.3Mb 10ms DropTail
$ns duplex-link $n1 $n2 0.3Mb 10ms DropTail
$ns duplex-link $n2 $n3 0.3Mb 10ms DropTail
$ns duplex-link $n1 $n4 0.3Mb 10ms DropTail
$ns duplex-link $n3 $n5 0.5Mb 10ms DropTail
$ns duplex-link $n4 $n5 0.5Mb 10ms DropTail
#Give node position (for NAM)
$ns duplex-link-op $n0 $n1 orient right
$ns duplex-link-op $n1 $n2 orient right
$ns duplex-link-op $n2 $n3 orient up
$ns duplex-link-op $n1 $n4 orient up-left
$ns duplex-link-op $n3 $n5 orient left-up
$ns duplex-link-op $n4 $n5 orient right-up
#Setup a TCP connection
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n5 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
```

```
#Setup a FTP over TCP connection  
set ftp [new Application/FTP]
```

```
$ftp attach-agent $tcp
```

```
$ftp set type _FTP
```

```
$ns rtmodel-at 1.0 down $n1 $n4
```

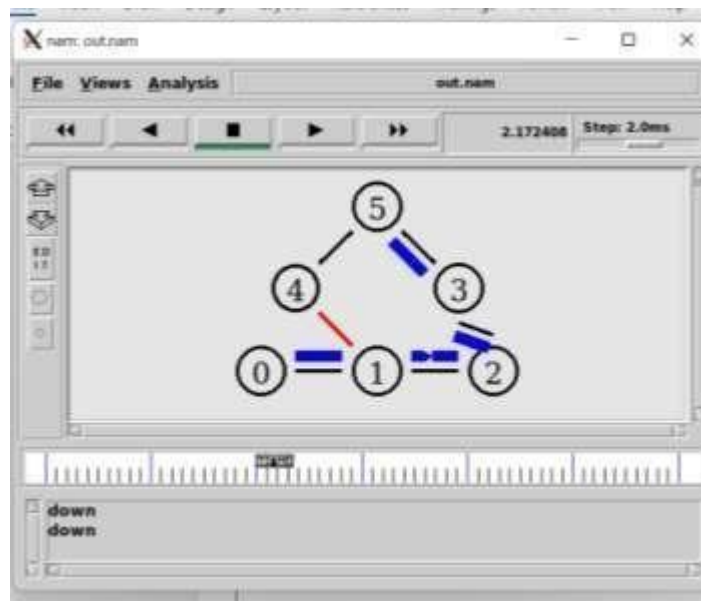
```
$ns rtmodel-at 4.5 up $n1 $n4
```

```
$ns at 0.1 "$ftp start"
```

```
$ns at 6.0 "finish"
```

```
$ns run
```

OUTPUT



RESULT

Performance evaluation of Routing protocols was analyzed using Simulation tool.

EXP NO: 4d**SIMULATION OF CONGESTION CONTROL AND FLOODING****DATE:****AIM:**

To write and execute a Java program to simulate Flooding and Congestion Control mechanisms in computer networks.

ALGORITHM:

1. Start the program.
2. Read the number of nodes and the source node from the user.
3. Initialize a queue and visited array for flooding simulation.
4. Begin flooding from the source node and forward packets to all unvisited nodes.
5. Display the order in which packets are forwarded to nodes.
6. Read buffer size and number of incoming packets from the user.
7. Compare incoming packets with buffer size to detect congestion.
8. If packets exceed buffer size, display packet drop information; otherwise, allow all packets.
9. Display congestion control results.
10. Stop the program.

PROGRAM:

```
import java.util.*;

class CongestionAndFlooding {

    static void flooding(int nodes, int source) {
        boolean[] visited = new boolean[nodes]; Queue q
        = new LinkedList<>(); q.add(source);
        visited[source] = true;

        System.out.println("\n--- Flooding Simulation ---");
        System.out.println("Source Node: " + source); while
        (!q.isEmpty()) {
            int current = q.poll();

            System.out.println("Packet forwarded to node: " + current); for (int
            i = 0; i < nodes; i++) {
                if (!visited[i]) {
```

```
        visited[i] = true;
        q.add(i);
    }
}

}

static void congestionControl(int bufferSize, int packets) {
    System.out.println("\n--- Congestion Control Simulation ---");
    System.out.println("Buffer Size: " + bufferSize);
    System.out.println("Incoming Packets: " + packets);
    if (packets <= bufferSize) {
        System.out.println("No Congestion detected."); System.out.println("All
        packets transmitted successfully.");
    }
    else {
        System.out.println("Congestion detected!");
        System.out.println("Packets transmitted: " + bufferSize);
        System.out.println("Packets dropped: " + (packets - bufferSize));
    }
}

public static void main(String[] args) { Scanner
    se new Scanner(System.in);
    System.out.println("Simulation of Congestion Control and Flooding");
    System.out.print("Enter number of nodes: ");
    int nodes sc.nextInt();
    System.out.print("Enter source node: "); int
    source sc.nextInt(); System.out.print("Enter
    buffer size: "); int buffer sc.nextInt();
    System.out.print("Enter number of incoming packets: "); int
    packets sc.nextInt();

    flooding(nodes, source);

    congestionControl(buffer, packets);

    sc.close();
}
}
```

OUTPUT:

```
C:\Users\DHARSANA\OneDrive\Desktop>javac --release 8 CongestionAndFlooding.java

C:\Users\DHARSANA\OneDrive\Desktop>java CongestionAndFlooding
Simulation of Congestion Control and Flooding
Enter number of nodes: 5
Enter source node: 0
Enter buffer size: 4
Enter number of incoming packets: 7

--- Flooding Simulation ---
Source Node: 0
Packet forwarded to node: 0
Packet forwarded to node: 1
Packet forwarded to node: 2
Packet forwarded to node: 3
Packet forwarded to node: 4

--- Congestion Control Simulation ---
Buffer Size: 4
Incoming Packets: 7
Congestion detected!
Packets transmitted: 4
Packets dropped: 3
```

RESULT:

The program was successfully executed, demonstrating flooding based packet forwarding and effective Detection of network congestion.

EX NO: 4e**IMPLEMENTATION OF SUBNETTING****DATE:****AIM:**

To write and execute a Java program to determine the class of an IP address, find its subnet mask, and calculate the first and last IP addresses of the network block.

ALGORITHM:

1. Start the program.
2. Read the IP address from the user.
3. Extract the first octet of the IP address.
4. Determine the IP class (A, B, C, D, or E) based on the first octet.
5. Assign the corresponding default subnet mask.
6. Split the IP address and subnet mask into four octets.
7. Perform bitwise AND operation to calculate the network address.
8. Calculate the last IP address using bitwise operations.
9. Display the IP class, subnet mask, first IP address, and last IP address.
10. Stop the program

PROGRAM:

```
import java.io.*;
import java.net.InetAddress;
public class Subnet1 {
    public static void main(String[] args) throws IOException { System.out.println("ENTER IP:");
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); String ip =
    br.readLine();
    String checkclass = ip.substring(0, 3); int cc =
    Integer.parseInt(checkclass); String mask =
    null;
    if(cc>0) {
        if(cc<=127) {
            mask = "255.0.0.0";
            System.out.println("Class A IP Address");
```

```

        System.out.println("SUBNET MASK:\n"+mask);
    }
    if(cc>=128 && cc<=191) { mask =
        "255.255.0.0";
        System.out.println("Class B IP Address");
        System.out.println("SUBNET MASK:\n"+mask);
    }
    if(cc>=192 && cc<=223) { mask =
        "255.255.255.0";
        System.out.println("Class C IP Address");
        System.out.println("SUBNET MASK:\n"+mask);
    }
    if(cc>=224 && cc<=239) { mask =
        "255.0.0.0";
        System.out.println("Class D IP Address Used for multicasting");
    }
    if(cc>=240 && cc<=254) { mask =
        "255.0.0.0";
        System.out.println("Class E IP Address Experimental Use");
    }
}

String networkAddr="";
String lastAddr="";
String[] ipAddrParts=ip.split("\\."); String[]
maskParts=mask.split("\\."); for(int
i=0;i<4;i++){
    int x=Integer.parseInt(ipAddrParts[i]); int
    y=Integer.parseInt(maskParts[i]); int z=x&y;
    networkAddr+=z+"."; int
    w=z/(y^255);
    lastAddr+=w+".";
    System.out.println("First IP of block: "+networkAddr); System.out.println("Last IP of block: "+lastAddr);
}
}

```

OUTPUT:

```

C:\Users\DHARSANA\OneDrive\Desktop>javac --release 8 Subnet1.java
C:\Users\DHARSANA\OneDrive\Desktop>java Subnet1
ENTER IP:
192.168.1.10
Class C IP Address
SUBNET MASK:
255.255.255.0
First IP of block: 192.168.1.0.
Last IP of block: 192.168.1.255.
C:\Users\DHARSANA\OneDrive\Desktop>

```

RESULT:

The subnet program was successfully executed, and the IP class, subnet mask, first IP address, and last IP address were correctly determined.

Exp. No: 5a	IMPLEMENTATION OF CYCLIC REDUNDANCY CHECK (CRC) FOR ERROR DETECTION
Date:	

AIM

To implement error detection and error correction techniques.

ALGORITHM

1. Open Turbo c++ software and type the program for error detection
2. Get the input in the form of bits.
3. Append 16 zeros as redundancy bits.
4. Divide the appended data using a divisor polynomial.
5. The resulting data should be transmitted to the receiver.
6. At the receiver the received data is entered.
7. The same process is repeated at the receiver.
8. If the remainder is zero there is no error otherwise there is some error in the received bits
9. Run the program.

C PROGRAM

```
#include<stdio.h>
char
m[50],g[50],r[50],q[50],temp[50];
void caltrans(int);
void
crc(int);
void
calram();
void
shiftl();
int main()
{
int n,i=0;
char ch,flag=0;
printf("Enter the frame bits:");
while((ch=getc(stdin))!='\n')
m[i++]=ch;
n=i;
for(i=0;i<16;i++)
m[n++]='0';
m[n]='\0';
printf("Message after appending 16 zeros:%s",m);
for(i=0;i<=16;i++)
g[i]='0';
g[0]=g[4]=g[11]=g[16]='1';g[17]='\0';
printf("\ngenerator:%s\n",
g); crc(n);
printf("\n\nquotient:%s",q)
; caltrans(n);
printf("\ntransmitted frame:%s",m);
printf("\nEnter transmitted frame:");
```

```
scanf("\n%s",m);
printf("CRC checking\n"); crc(n);
printf("\n\nlast remainder:%s",r);
for(i=0;i<16;i++)
    if(r[i]!=
    '0')
    flag=1; else
printf("Error during transmission"); else
printf("\n\nReceived freme is correct");
}

void crc(int n)
{
    int i,j;
    for(i=0;i<n;i++)
        temp[i]=m[i];
    for(i=0;i<16;i++)
        r[i]=m[i];
    printf("\nintermediate
    remainder\n"); for(i=0;i<n-16;i++)
    {
        if(r[0]=
        '1')
        {
            q[i]
            ='1'
        }
        ;
        calram();
    }
    else
    {
        q[i]
        ='0'
    }
    ;
    shiftl();
}
r[16]=m[17
+i];
r[17]='\0';
printf("\nremainder
%d:%s",i+1,r);
for(j=0;j<=17;j++)
    temp[j]=r[j];
}
q[n-16]='\0';
}
void calram()
{
    int i,j;
    for(i=1;i<=16;i++)
```

```
)
r[i-1]=((int)temp[i]-48)^((int)g[i]-48)+48;
}
printf("\nEnter transmitted
frame:"); scanf("\n%s",m);

printf("CRC checking\n");
crc(n); printf("\n\nlast
remainder:%s",r);
for(i=0;i<16;i++)
if(r[i]!=
'0')
flag=1;
else
continue;
if(flag
==1)
printf("Error during transmission");
else
printf("\n\nReceived freme is correct");
}
void crc(int n)
{
int i,j;
for(i=0;i<n;i++)
temp[i]=m[i];
for(i=0;i<16;i++)
r[i]=m[i];
printf("\nintermediate
remainder\n");

for(i=0;i<n-16;i++)
{
if(r[0]!=
'1')
{
q[i]
='1'
;
calram();
}
else
{
q[i]
='0'
;
shiftl();
```

```

}
r[16]=m[17
+i];
r[17]='\0';
printf("\nremainder
%d:%s",i+1,r);
for(j=0;j<=17;j++)
temp[j]=r[j];
}
q[n-16]='\0';
}
void calram()
{
int i,j;
for(i=1;i<=16;i++)
)
r[i-1]=((int)temp[i]-48)^((int)g[i]-48)+48;
}
void shiftl()
{
int i;
for(i=1;i<=16;i
++) r[i-1]=r[i];
}
void caltrans(int n)
{
int i,k=0;
for(i=n-16;i<n;i++)
m[i]=((int)m[i]-48)^((int)r[k++]-48)+48;
m[i]='\0';
}

```

OUTPUT

Enter the Frame Bits:

1011

The msg after appending 16 zeros:

10110000000000000000

The Transmitted frame

is:10111011000101101011 Enter the

transmitted Frame **10111011000101101011**

Received

msg:**10111011000101101011** The

Remainder is:**0000000000000000**

Received frame is correct.

RESULT

Thus the error detection and error correction